KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 4 Quiz

React I: Bileşenler

Doç. Dr. Tolga Berber · https://avesis.ktu.edu.tr/tberber

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (6 puan)

4 • Değerlendirme Hedef 1

React ile Hafta 3'ün elle DOM güncelleyen kodu arasındaki fark hakkında hangisi doğrudur?

(a) React'te yalnızca "durum ne ise görünüm ne olmalı" tanımlanır (bildirimsel model); DOM'un hangi adımlarla güncelleneceğine React kendisi karar verir. Hafta 3'te ise `querySelector`/`classList.toggle` gibi adımları siz elle yazıyordunuz (zorunlu model).

(b) React de tıpkı Hafta 3 gibi DOM güncelleme adımlarını (`querySelector`, `classList.toggle` vb.) doğrudan sizin yazmanızı gerektirir; tek fark sözdizimidir.

(c) Hafta 3'ün elle DOM kodu bildirimsel, React ise zorunludur; React'te durum değiştiğinde hangi DOM düğümünün güncelleneceğini siz adım adım belirtirsiniz.

(d) İkisi de aynı bildirimsel modeli kullanır; tek fark React'in TypeScript, Hafta 3'ün düz JavaScript kullanmasıdır.

Soru 2 (6 puan)

4 • Değerlendirme Hedef 1

Aşağıdaki TSX parçacıklarından hangisi `tsc`/Vite tarafından hatasız derlenir?

(a) `<p>{if (sayi > 0) { <span>Pozitif</span> } }</p>` — süslü parantez içine bir `if` deyimi yazılmıştır.

(b) `<p>{for (let i = 0; i < 3; i++) { i } }</p>` — süslü parantez içine bir `for` döngüsü yazılmıştır.

(c) `<p>{sayi > 0 ? 'Pozitif' : 'Negatif'}</p>` — süslü parantez içinde yalnızca bir değer üreten üçlü operatör ifadesi vardır.

(d) `<p>{let x = sayi + 1;}</p>` — süslü parantez içine bir `let` değişken tanımı yazılmıştır.

Soru 3 (8 puan)

4 • Değerlendirme Hedef 1

Aşağıdaki kesit `src/main.tsx` dosyasından alınmıştır.

```
// createRoot ile App bileşenini #root'a, StrictMode içinde bağlar.
createRoot(document.getElementById('root')!).render(
  <StrictMode>
    <App />
  </StrictMode>,
```

Soru 3 (devamı)

```
);
```

(a) Bu kod tam olarak ne yapar? (b) `index.html` içindeki `<div id="root"></div>` öğesi (bu kod değişmeden) kaldırılsaydı ne olurdu — bu bir `tsc` derleme hatası mı, yoksa çalışma zamanı (*runtime*) hatası mı olurdu?

Soru 4 (10 puan)

4 • Değerlendirme Hedef 1

Sanal DOM ve uzlaştırma (*reconciliation*) kavramlarını kullanarak, `Koltuk` bileşenindeki bir `setDurum` çağrısının ardından React'in neden sayfanın tamamını yeniden çizmediğini açıklayınız. Bunu Hafta 3'ün `koltuk_tiklama` fonksiyonunun tek bir DOM düğümünü elle bulup değiştirmesiyle (`querySelector` + `classList.toggle`) karşılaştırınız.

Soru 5 (6 puan)

4 • Değerlendirme Hedef 2

`EtkinlikKarti` bileşeni `{ etkinlik }: EtkinlikKartiProps` parametresini alır. `props` (burada `etkinlik`) hakkında hangisi doğrudur?

(a) `props`, bir bileşene yalnızca sayısal (`number`) değerler geçirmek için kullanılır; nesne/dizi tipinde bir `props` tanımlanamaz.

(b) `props`, bir üst bileşenin bir alt bileşene veri geçirmesinin tek yoludur; bir fonksiyona parametre geçirmeye benzer ama salt okunur kabul edilir — `EtkinlikKarti`, kendisine gelen `etkinlik` prop'unu doğrudan değiştirmemelidir.

(c) `props`, bir bileşenin kendi `useState`'i ile aynı mekanizmadır; ikisi de bileşenin kendi içinde tutulan, dışarıdan görünmeyen veridir.

(d) Bir alt bileşen, kendisine gelen bir `props` değerini değiştirdiğinde bu değişiklik otomatik olarak üst bileşenin verisine de yansır; bu, React'in "iki yönlü veri akışı" özelliğidir.

Soru 6 (6 puan)

4 • Değerlendirme Hedef 2

`Bolum` bileşeni `{ baslik, children }: BolumProps` şeklinde tanımlanmıştır ve `<Bolum baslik="Etkinlikler"><EtkinlikListesi ... /></Bolum>` olarak kullanılır. `children` hakkında hangisi doğrudur?

(a) `children`, `Bolum` bileşeninin kendi içinde tanımladığı sabit bir JSX'tir; açılış/kapanış etiketleri arasına ne yazılırsa yazılsın hep aynı içerik gösterilir.

Soru 6 (devamı)

(b) `children` yalnızca metin (`string`) içerebilir; bir bileşenin açılış/kapanış etiketleri arasına başka bir bileşen (ör. `<EtkinlikListesi />`) yerleştirilemez.

(c) `children` prop'u yalnızca sınıf (*class*) bileşenlerinde çalışır; fonksiyon bileşenlerinde `children` kullanmak için ek bir kütüphane kurulması gerekir.

(d) `children`, `Bolum`'un açılış/kapanış etiketleri arasına yazılan JSX'i (burada `<EtkinlikListesi ... />`) otomatik olarak `props.children` şeklinde `Bolum`'a taşır; `Bolum` bu içeriği kendisi üretmez, yalnızca nereye yerleştireceğine (başlığın altına) karar verir.

Soru 7 (8 puan)

4 • Değerlendirme Hedef 2

Aşağıdaki kesitler `src/bilesenler/EtkinlikKarti.tsx` dosyasından alınmıştır.

```tsx
interface EtkinlikKartiProps {
  etkinlik: Etkinlik;
}
```

```tsx
// Hf.2'nin article.etkinlik-karti HTML yapısının React karşılığı;
// biletFiyati 0 ise üçlü operatörle "Ücretsiz" rozeti gösterilir.
export function EtkinlikKarti({ etkinlik }: EtkinlikKartiProps) {
  return (
    <article className="etkinlik-karti">
      <h3>{etkinlik.baslik}</h3>
      <p>
        {etkinlik.tarih} · {etkinlik.mekan}
      </p>
      <p>
        Kapasite: {etkinlik.kapasite} ·{' '}
        {etkinlik.biletFiyati === 0 ? (
          <span className="rozet-ucretsiz">Ücretsiz</span>
        ) : (
          `Bilet: ${etkinlik.biletFiyati} TL`
        )}
      </p>
    </article>
  );
}
```

(a) `veri.ts`teki `'Kariyer Günleri'` etkinliği (`biletFiyati: 0`) bu bileşene `etkinlik` prop'u olarak geçirildiğinde fiyat satırında hangi metin görünür ve neden? (b) `App.tsx` içinde yanlışlıkla `<EtkinlikKarti etkinlik="Bahar Konseri" />` (bir `string`) yazarsanız ne olur?

Soru 8 (10 puan)

4 • Değerlendirme Hedef 2

`App` → `EtkinlikListesi` → `EtkinlikKarti` bileşen ağacında `props`'un yalnızca yukarıdan aşağıya aktığını biliyorsunuz. `EtkinlikKarti`, gösterdiği etkinliğin fiyatını kendisi değiştirmek isteseydi (ör. bir indirim uygulamak için) bunu neden doğrudan yapamayacağını ve bunun ne gibi bir soruna yol açacağını kendi cümlelerinizle açıklayınız.

Soru 8 (devamı)

Soru 9 (6 puan)

4 • Değerlendirme Hedef 3

`const [durum, setDurum] = useState<KoltukDurumu>(koltuk.durum);` satırından sonra bir `onClick` işleyicisi içinde `durum = 'secili';` (doğrudan atama) yazarsanız ne olur?

(a) `tsc` bir derleme hatası verir; `useState`'in döndürdüğü ilk eleman `const` bağlamlıdır, yeniden atanamaz — state'i değiştirmenin tek yolu ikinci elemanı (`setDurum`) çağırmaktır.

(b) Kod sorunsuz derlenir ve çalışır; ekran anında `'secili'` durumunu yansıtır çünkü React her değişkeni otomatik olarak izler.

(c) `durum` değişkeni değişir ve React bunu bir sonraki render sırasında fark edip ekranı günceller; yalnızca bir kare (*frame*) gecikme yaşanır.

(d) `tsc` yalnızca bir uyarı (*warning*) verir ama derleme yine de `dist/` klasörüne başarıyla çıktı üretir; sorun yalnızca çalışma zamanında ortaya çıkar.

Soru 10 (6 puan)

4 • Değerlendirme Hedef 3

`Koltuk` bileşeninde `durum` alanı `'dolu'` olan bir koltuğa tıklandığında ne olur?

(a) `setDurum` çağrılır ve `durum` `'dolu'`'dan `'secili'`'ye geçer; ancak ekran görsel olarak değişmez çünkü CSS `dolu` sınıfını önceliklendirir.

(b) `onClick` işleyicisi hiç atanmaz; React, `dolu` durumundaki bir bileşende olay dinleyicisi hiç kurmaz.

(c) `button` öğesi `disabled={durum === 'dolu'}` olduğundan tarayıcı tıklamayı zaten ateşlemez; ayrıca `tikla` fonksiyonu içindeki `if (durum === 'dolu') return;` satırı da aynı durumu ele alan ikinci bir güvenlik katmanıdır — her iki durumda da `durum` değişmez.

(d) `tsc` bu koltuğu derleme zamanında tespit edip programı çalıştırmayı reddeder; `dolu` bir koltuk tanımlamak başlı başına bir tip hatasıdır.

Soru 11 (8 puan)

4 • Değerlendirme Hedef 3

Aşağıdaki kesitler `src/bilesenler/Koltuk.tsx` dosyasından alınmıştır.

```
1 // Koltuğun kendi durumu, koltuk.durum başlangıç değeriyle yerel bir
2 // useState'te tutulur; App ya da KoltukPlani bu değeri GÖRMEZ.
3 const [durum, setDurum] = useState<KoltukDurumu>(koltuk.durum);
4
5 function tikla() {
6   if (durum === 'dolu') return;
7   setDurum((onceki) => (onceki === 'bos' ? 'secili' : 'bos'));
```

Soru 11 (devamı)

```
}
```

(a) `tikla` fonksiyonu içinde `setDurum((onceki) => (onceki === 'bos' ? 'secili' : 'bos'));` şeklinde bir güncelleyici fonksiyon kullanılmıştır; bunun `setDurum(durum === 'bos' ? 'secili' : 'bos')` yazmaktan farkı nedir (değişmezlik ilkesiyle ilişkilendirerek açıklayın)? (b) `durum` alanı `'bos'` olan bir koltuğa arka arkaya iki kez tıklandığında `durum` hangi sırayla değişir?

Soru 12 (10 puan)

4 • Değerlendirme Hedef 3

Değişmezlik (*immutability*) ilkesinin `useState` ile neden zorunlu olduğunu, Hafta 3 Ünite 1'in referans semantiği ve C/C++'ın `const` disipliniyle karşılaştırarak açıklayınız. Bir state değerini doğrudan değiştirirseniz (mutasyon) React'in bunu neden "fark etmeyeceğini" anlatınız.

Soru 13 (6 puan)

4 • Değerlendirme Hedef 4

`koltuklar.map((koltuk, index) => <Koltuk key={index} koltuk={koltuk} />)` şeklinde, veriden gelen `koltuk.id` yerine dizinin sıra numarasını (`index`) `key` olarak kullanırsanız, `koltuklar` dizisinin başına yeni bir koltuk eklendiğinde ne olabilir?

(a) Hiçbir sorun çıkmaz; `index` de `id` gibi her koltuk için kararlı ve benzersizdir, React ikisi arasında fark gözetmez.

(b) React, listedeki her koltuğu artık yanlış `key`'lerle eşleyebilir — yeni koltuk eklenince tüm sonraki koltukların `index`'i bir kayar; bu da React'in "bu hangi koltuktu" sorusunu yanlış cevaplamasına ve bir koltuğun `'secili'` gibi kendi state'inin yanlış bir koltukta görünmesine yol açabilir.

(c) `tsc`, `index`'i `key` olarak kullanmayı bir derleme hatası olarak işaretler; kod hiç derlenmez.

(d) React otomatik olarak `index` yerine `koltuk.id`'yi kullanır; `key` prop'unun değeri her zaman önemsizdir.

Soru 14 (8 puan)

4 • Değerlendirme Hedef 4

Aşağıdaki kesit `src/bilesenler/KoltukPlani.tsx` dosyasından alınmıştır; `veri.ts`'teki 12 koltuktan `id: 6` (B2) ve `id: 12` (C4) `'dolu'` olarak işaretlidir.

Soru 14 (devamı)

```tsx
// Boş dizi için üçlü operatörle mesaj; aksi halde map+key ile Koltuk
// ızgarası üretilir.
export function KoltukPlani({ koltuklar }: KoltukPlaniProps) {
  return koltuklar.length === 0 ? (
    <p>Bu etkinlik için koltuk tanımlı değil.</p>
  ) : (
    <div className="koltuk-plani">
      {koltuklar.map((koltuk) => (
        <Koltuk key={koltuk.id} koltuk={koltuk} />
      ))}
    </div>
  );
}
```

(a) `koltuklar` dizisi boş olsaydı hangi JSX döner ve bu, hangi koşullu render kalıbını kullanır — `EtkinlikListesi`'nin erken dönüş kalıbından farkı nedir? (b) `key={koltuk.id}` yerine `key={index}` kullanılsaydı ve `koltuklar` dizisinin başına yeni bir koltuk eklenseydi, B2 (`id: 6`) koltuğunun `'dolu'` rozeti ekranda hangi somut hataya yol açabilirdi?

Soru 15 (10 puan)

4 • Değerlendirme Hedef 4

`etkinlikler.map((etkinlik) => <EtkinlikKarti key={etkinlik.id} etkinlik={etkinlik} />)` satırından `key={etkinlik.id}` kısmını tamamen silerseniz kod hâlâ çalışır mı? Tarayıcı konsolunda ne görürsünüz ve `key` prop'unun React'in uzlaştırma sürecindeki rolünü, bir vestiyerdeki numaralı fişler benzetmesini kullanarak açıklayınız.